

UNIVERSIDADE D
COIMBRA



Sistemas Distribuídos

Licenciatura em Engenharia Informática

Googol

Motor de pesquisa de páginas web distribuído

Relatório de Projeto
Meta 1

Diogo Rodrigues | **2022257625**

Guilherme Rosmaninho | **2022257636**

PL6

2025/2026

Testes de *Software*

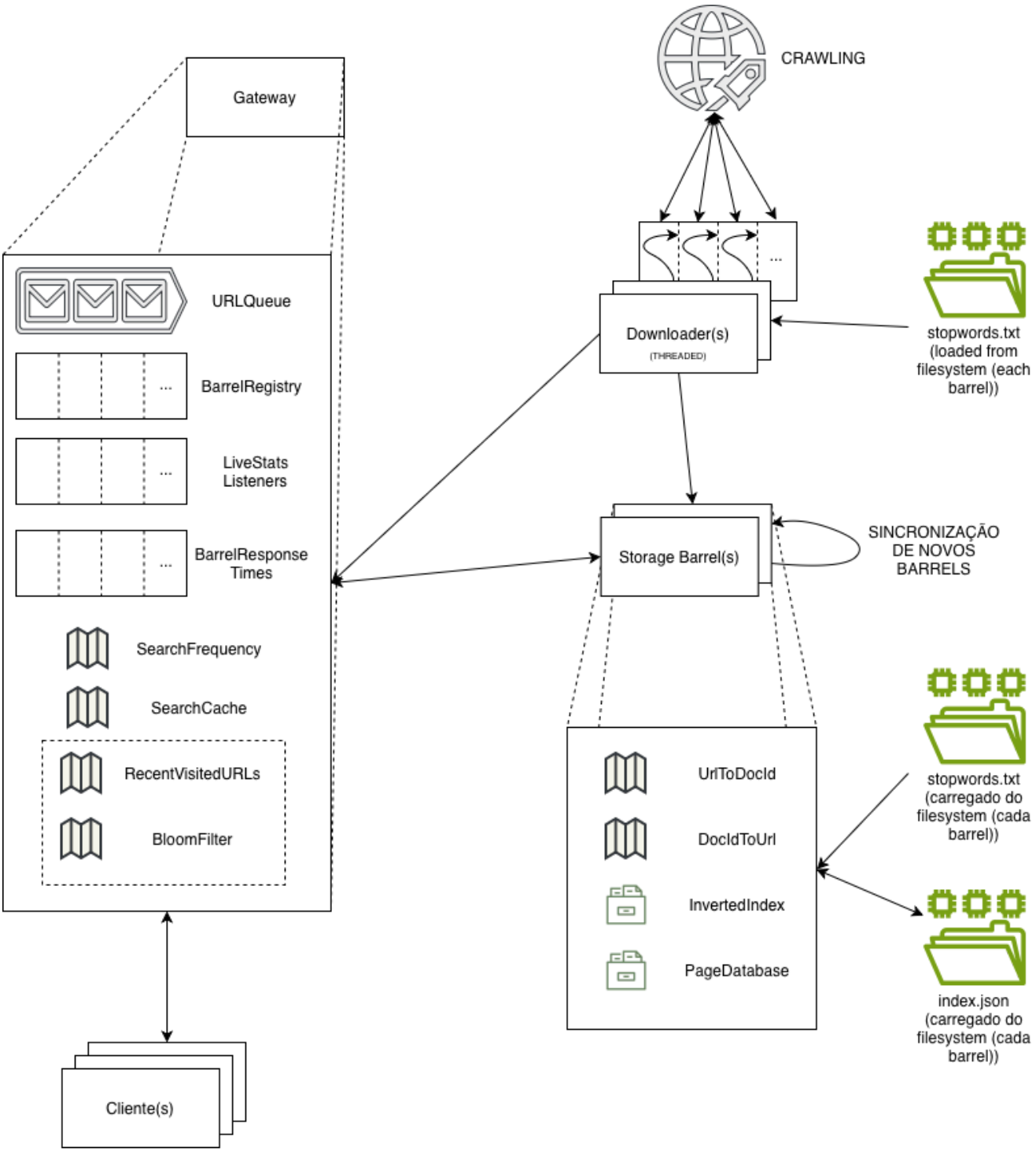
Teste	Resultado
Indexar novo *URL* introduzido por utilizador	PASS
Indexar iterativamente ou recursivamente todos os *URLs* encontrados	PASS
Pesquisar todas as páginas que contenham um conjunto de palavras	PASS
Pesquisar páginas ordenadas por número de ligações recebidas de outras páginas	PASS
Consultar lista de páginas com ligações para uma página específica	PASS
Página de estatísticas atualizada em tempo real	PASS
Resultados de pesquisa agrupados de 10 em 10	PASS
Aprendizagem computacional de distribuída de palavras vazias	N/A
Informação idêntica em todos os *storage barrels* (*reliable multicast*)	PASS
Serviço correto se funcionar pelo menos um *storage barrel* e *gateway*	PASS
Os *storage barrels* recuperam o seu estado se avariarem (*crash*)	PASS
Balanceamento de carga nas pesquisas sobre os *storage barrels*	PASS
Os *downloaders* executam em paralelo	PASS
A *gateway* recupera de quaisquer avarias internas (não tem efeito nos clientes)	PASS
Pedidos de indexação respondidos por apenas um *downloader*	PASS

Extras

Aprendizagem computacional de distribuída de palavras vazias	NÃO IMPLEMENTADO
Uso de estruturas avançadas tais como filtro de *Bloom* e *DocIDs*	PASS
Pesquisas avançadas e melhores algoritmos de *ranking* de páginas	PASS
Novo *storage barrel* sincroniza-se com os existentes	PASS

Arquitetura do sistema

No diagrama abaixo, estão presentes os componentes do sistema, acompanhados de uma seleção das principais estruturas de dados internas.



Estrutura de ficheiros

Presente no README.md.

Detalhes de funcionamento

Nesta secção estão presentes os principais detalhes de implementação e funcionalidades de destaque associados ao presente sistema.

- Funcionamento implementação do *RPC/RMI* nos diversos componentes:

O *gateway*, sendo o cérebro do funcionamento do sistema, guarda diversas estruturas essenciais para o funcionamento do sistema como um todo. Entre elas está o *BarrelRegistry*, uma estrutura que permite o fácil acesso por todos os componentes aos *StorageBarrels* conhecidos pelo gateway. O *BarrelRegistry*, ainda que faça parte do mesmo processo que o *Gateway*, é uma unidade/objeto independente que facilita, nomeadamente, a comunicação *Downloader*→Barrels (via multicast). Esta separação não seria estritamente necessária, mas considera-se que vantajosa ao nível organizacional e modular.

Apenas os *storage barrels e a gateway* (segundo a gateway e o *barrel registry*) é que se encontram registados como servidores *RMI*, sendo todos os componentes clientes *RMI*.

Javadocs sobre os métodos remotos disponibilizados no código fonte.

- Funcionamento implementação da replicação de informação nos *storage barrels* (via *reliable multicast*):

A confiabilidade do *reliable multicast* implementado assenta numa lógica de tentativa de retransmissão. Ao processar uma página, o *downloader* vai recorrer efetuar o *multicast* para todos os *storage barrels* levando a que exista a replicação natural da informação por todos os barris ativos no sistema. A deteção de uma *exception* na tarefa de indexação das páginas levará à subsequente transmissão da página em causa. Eventuais insucessos resultantes de sucessivas falhas na retransmissão serão apresentadas ao utilizador e os barris não responsivos são automaticamente removidos do *BarrelRegistry,* assumindo-se que cessaram atividade desde a última verificação de estado.

- Funcionamento e implementação da sincronização de novos barrels:

Um *barrel* novo, que se junte ao sistema em *runtime*, irá verificar o tamanho dos índices dos restantes *peers* e sincronizar-se com o maior (caso exista). A sincronização é feita em *chunks*, valor que é parametrizável no ficheiro *system.properties*. O estado deste processo é atualizado e é visível para todos os clientes através da funcionalidade de sincronização em tempo real. A sincronização é feita numa thread separada, de forma a não impactar o funcionamento dos dois barrels envolvidos (numa dada operação de sincronização), permitindo que continuem a atualizar o seu índice de forma paralela (a

sincronização apenas envolve as páginas indexadas antes do instante de lançamento do novo *barrel*.

- Detalhes de implementação de *load balancing* dos storage barrels efetuado pela *gateway*:

  O *load balancing* dos *storage barrels* é feito através da constante determinação do tempo médio de resposta de cada um. Através desta métrica efetua-se a seleção do *storage barrel* com o menor tempo médio de resposta, a cada *index request.* Adicionalmente, como *fallback*, se algo correr mal nesta determinação, é utilizado, como fallback, o algoritmo *round robin*.

- Detalhes de implementação de um algoritmo de *ranking* de páginas avançado (TF-IDF):

Devido à boa relação entre dificuldade de implementação e desempenho de ranking de páginas, escolheu-se a implementação do algoritmo TF-IDF. A ideia central deste algoritmo, é atribuir um score maior às páginas onde um termo aparece muitas vezes *(TF - term frequency)* em conjugação uma métrica que avalia a raridade desse termo no conjunto dos documentos (*IDF - inverse document frequency*).

$$TF(t,d) = \frac{number\ of\ times\ t\ appears\ in\ d}{total\ number\ of\ terms\ in\ d}$$

$$IDF(t) = log\frac{N}{1+df}$$

$$TF - IDF(t,d) = TF(t,d) * IDF(t)$$

Estes *scores* são determinados pelo storage barrel a cada pesquisa. Como critério de desempate, manteve-se o ranking por número de *inlinks* (referências de outras páginas).

- Detalhes da funcionalidade de estatísticas em tempo real para os clientes:

A funcionalidade de estatísticas consiste num mecanismo de subscrição, onde clientes interessados se registam e passam a receber estas atualizações fornecidas pelo gateway. Para cada métrica sujeita a monitorização (ex. top 10 pesquisas), o método envolvido será o gatilho para que seja enviada uma atualização de todos os clientes registados como *listeners*.

- Implementação do filtro de *Bloom*:

O filtro de *bloom* incluído no sistema recorre a uma biblioteca java chamada Google Guava. Os parâmetros desta estrutura são alteráveis através do ficheiro de configurações do sistema. De forma a combater os falsos positivos do bloom filter foi

implementada uma cache *LRU* (*least recently used*, remove o item menos utilizado), onde um determinado valor de últimos *urls* visitados. Perante uma nova *query*, é verificado se o url está na cache. A sua presença na cache dá-nos 100% de certeza que o *URL* já foi visitado. Perante um *miss* nesta *cache*, o *URL* é finalmente submetido ao filtro de *Bloom*, onde um resultado negativo é 100% confiável, mas pelo menos agora, estamos mais protegidos dos seus falsos positivos (com o auxílio da cache). Esta estrutura aumenta significativamente a eficiência de memória e velocidade de resposta, eliminando a necessidade de fazer pesquisas custosas por todos os registos de *URLs* indexados.

- Implementação de *DocIDs* para redução de memória utilizada (*tradeoff* por maior *overhead* de processamento):

No sistema desenvolvido, a implementação de *DocIDs* funciona da seguinte forma: chega uma nova página a um *storage barrel,* determina-se o seu *DocID* (atribui-se um número único à página recém chegada)*,* convertem-se as páginas referenciadas pela mesma para *DocIDs* (de forma semelhante), e aqui, efetua-se então a criação de placeholders dos outlinks (da original), para as páginas ainda por indexar. Estes placeholders são entradas incompletas, onde apenas se preenche o campo dos *inlinks* da mesma, isto porque, sem esta operação, as páginas indexadas no futuro não iriam ter nos seus registos as referências de páginas já indexadas.

De forma inversa, o acesso a páginas indexadas no barril é feito através da tradução dos *DocIDs,* para *URLs*. Estas operações são transparentes para qualquer componente fora do *storage barrel*. Esta técnica permite uma menor utilização de memória (uma vez que *URLs* repetidos são substituídos por *DocID*, utilizando muito menos recursos, requirindo no entanto, mais overhead de processamento, dada a necessidade de efetuar a tradução de *DocID* para *URL* e vice-versa.

- Tratamento de *stopwords* (*Downloader* e *Storage Barrel*):

O tratamento de *stopwords* é feito de forma simples, sendo os dois componentes envolvidos o *Downloader* e o *Storage Barrel.* O downloader processa as palavras das páginas exploradas e remove diretamente as palavras vazias importadas de um ficheiro de texto (offload de algum trabalho para o barrel). No storage barrel, é efetuada a remoção das palavras vazias de cada pesquisa, de forma similar ao *Downloader*.

- Gestores de conexão contínuos nos diferentes componentes:

  Os diferentes componentes do sistema estão implementados de forma a efetuarem uma verificação contínua da conexão com o *gateway*. Desta forma, estão resilientes perante a sua falha e assim que possível, efetuarão a reconexão para continuar a normal atividade assim que possivel.

- Outros pontos diversos:
  - Priority queue para que pesquisas de utilizadores sejam priorizadas em relação aos *URLs crawled*.
  - A cada início de execução, os *barrels* carregam um ficheiro *.json* (para simplicidade) onde estão todas as indexações fruto do último armazenamento das mesmas, este que, é feito no encerramento dos mesmos.

Distribuição de tarefas

A distribuição de tarefas efetuou-se da seguinte forma (de uma maneira geral):

Diogo:

- Indexação recursiva de *URLs* encontrados (*crawled*)
- Pesquisa de páginas que contenham um conjunto de termos
- Pesquisa de páginas que referenciam um URL
- Resultados ordenados por importância; TF-IDF
- Implementação de DocIDs
- Estatísticas em tempo real
- Tratamento de stopwords
- Load Balancing
- Sincronização de novos barrels adicionados ao sistema
- Multi threading no downloaders

Guilherme:

- Filtro de Bloom
- Indexação de um novo URL
- Reliable Multicast
- Tratamento das Stop Words
- Pesquisa por páginas com um conjunto de palavras
- Desenho do Cliente